

MLatom

AI-enhanced computational chemistry



AIQM1

AIQM1 (artificial intelligence–quantum mechanical method 1) is a general-purpose method approaching the gold-standard coupled cluster quantum mechanical method with high computational speed of the approximate low-level semiempirical quantum mechanical methods for the ground-state, closed-shell species, but also transferable for calculation of charged and radical species as well as for excited-state calculations with a good accuracy. See AIQM1 paper for more details:

*Peikun Zheng, Roman Zubatyuk, Wei Wu, Olexandr Isayev, Pavlo O. Dral, [Artificial Intelligence-Enhanced Quantum Chemical Method with Broad Applicability](#), Nat. Commun., **2021**, 12, 7022, DOI: 10.1038/s41467-021-27340-2*

Please cite this paper in the publications using the AIQM1 method.

Strengths: AIQM1 is especially good for energy calculations and geometry optimizations of closed-shell molecules in their ground-state.

Limitations: This method is currently limited to compounds only containing H, C, N, and O elements.

Availability: AIQM1 is available in MLatom package and most calculations can be performed online on our [MLatom@XACS cloud computing service](#) without any need for programs installations. Alternatively, you can install MLatom and other required packages to which MLatom is interfaced. Among required packages are [TorchANI](#) and [dftd4](#). The

released MLatom version requires the [MNDO](#) program, while a special MLatom@XACS version for calculations on the cloud does not depend on it and currently uses a [development version](#) of [SCINE Sparrow](#) (see also a [paper](#) on Sparrow). Some features also require interfaces to the [Gaussian](#) or [ASE](#) packages. Installation instructions, a usage manual, and proper citations are given below.

AIQM1 is now also available in a [development version](#) of [SCINE Sparrow](#) independently from MLatom, see a [paper](#) on Sparrow.

Table of Contents



1. Installation

- 1.1. MLatom
- 1.2. MNDO
- 1.3. TorchANI
- 1.4. dftd4
- 1.5. Optional packages for additional features
 - 1.5.1. Gaussian
 - 1.5.2. ASE

2. Usage

- 2.1. Get started: Simplest calculation job
- 2.2. Geometry optimization
 - 2.2.1. Geometry optimization of transition states structures
- 2.3. Calculation of thermochemical properties
- 2.4. Beyond closed-shell molecules in ground state
 - 2.4.1. Calculations of charged species and radicals
 - 2.4.2. Vertical excitation energies
 - 2.4.3. Excited-state geometry optimization

3. Visualizing molecular orbitals

4. Citations

Installation

The easiest way is not to install anything at all and just use our [MLatom@XACS cloud computing service](#) which can support most of the types of calculations. Below are instructions if you need to install it on your computer (cluster).

MLatom

It is recommended to install the latest MLatom release as described in [download](#) page. See [manual](#) pages on this website for [license](#) and further instructions.

MNDO

MNDO program is required to provide the ODM2* part of AIQM1. Alternatively, the [development version](#) is used on the MLatom@XACS cloud.

The free binary and open-source code of the MNDO program is available from the official distributors of the MNDO code as described at <https://mndo.kofo.mpg.de>.

After the MNDO program is installed, you need to set the environmental variable pointing to the MNDO executable (typically `mndo99`), e.g., in bash:

```
export mndobin=[path to the executable]/mndo99
```

TorchANI

TorchANI program is required to provide the neural network (NN) part of AIQM1.

TorchANI is an open-source package. The latest version when writing this tutorial is v2.2.

Installation

1. install *Numpy* and nightly version of *PyTorch* (if you do not have them already):

```
pip install numpy
pip install --pre torch torchvision -f
https://download.pytorch.org/whl/nightly/cu100/torch_nightly.html
```

2. install TorchANI:

```
pip install torchani
```

Visit <https://aiqm.github.io/torchani/> for more info. The latest version of TorchANI used when writing this tutorial is v2.2, you can install this version by `pip install torchani==2.2` if there are any problems when running with the newest version of TorchANI. The CUDA extension for AEV calculation is not supported for the NN part of AIQM1 now.

dftd4

dftd4 program is required to provide the D4 part of AIQM1.

The dftd4 program can be [obtained](#) as both executable and open-source code. We recommend using [dftd4 v3.5.0](#) ([dftd4 v2.5.0](#) for the MLatom versions earlier than 3.0.1), which can calculate Hessian needed for thermochemical calculations. To install the dftd4 program from source code, please see the README.md file on dftd4 GitHub page for more details.

After the dftd4 program is installed, you need to set the environmental variable pointing to the dftd4 executable, e.g., in bash:

```
export dftd4bin=[path to the executable]/dftd4
```

Optional packages for additional features

To perform geometry optimizations or calculations of heats of formation, MLatom relies on either Gaussian or ASE package (any of them can be used; the first one is commercial, the second one is open-source).

Gaussian

Our implementation work with both Gaussian 09 and Gaussian 16. It is a commercial program, which can be [obtained and installed separately](#).

To use Gaussian interface, make sure that your environmental variable `GAUSS_EXEDIR` points to the right place.

ASE

The ASE (Atomic Simulation Environment) are Python modules, which can be installed as described on [ASE website](#).

Usage

To perform AIQM1 calculations, you can provide input files with appropriate options to MLatom (command line options also supported) and then run MLatom calculations as usual (see [manual](#) and [tutorial](#) pages on this website). Below we provide examples of some typical uses. In all cases, we use `mlatom` as alias to `$pathToMLatom/MLatom.py` (it is useful to setup such an alias in your shell). The examples below were tested with the interface to the MNDO program. If you use the [MLatom @XACS cloud computing service](#), it runs calculations using the interface to the SCINE Sparrow package, thus, the numbers may be slightly different.

Get started: Simplest calculation job

Single-point calculations of energies (and gradients if needed) of closed-shell molecules in electronic ground state is the simplest job, which can be run with 3-4 line MLatom input file, e.g., `sp.inp`:

```
AIQM1 # or AIQM1@DFT or AIQM1@DFT* if you want to use these methods
xyzfile=sp.xyz
yestfile=enest.dat
ygradxyzestfile=gradest.dat
```

This input requires a `sp.xyz` file with XYZ geometries of molecules (you can provide many molecules as usual for MLatom), e.g., for hydrogen and methane `sp.xyz` file can look like (geometries in Å):

```
2

H      0.000000      0.000000      0.363008
H      0.000000      0.000000     -0.363008
```

5

C	0.000000	0.000000	0.000000
H	0.627580	0.627580	0.627580
H	-0.627580	-0.627580	0.627580
H	0.627580	-0.627580	-0.627580
H	-0.627580	0.627580	-0.627580

After you prepared your input files `sp.inp` and `sp.xyz`, you can run MLatom as usual:

```
mLatom sp.inp > sp.out
```

After the calculations finish, MLatom output `sp.out` will contain the standard deviation of NN prediction and components of AIQM1 energies:

```
Standard deviation of NN contribution : 0.00892407 Hartree
5.59994 kcal/mol
NN contribution : -0.00210898 Hartree
Sum of atomic self energies : -0.08587317 Hartree
ODM2* contribution : -1.09094119 Hartree
D4 contribution : -0.00000889 Hartree
Total energy : -1.17893224 Hartree
Standard deviation of NN contribution : 0.00025608 Hartree
0.16069 kcal/mol
NN contribution : 0.00958812 Hartree
Sum of atomic self energies : -33.60470494 Hartree
ODM2* contribution : -6.86968756 Hartree
D4 contribution : -0.00010193 Hartree
Total energy : -40.46490632 Hartree
```

You can also get AIQM1 energies saved in file `enest.dat` looking like this (energies in Hartree):

```
-1.178932238420  
-40.464906315250
```

and gradients saved in file `gradest.dat` looking like this (gradients in Hartree/Å):

2

0.000000000000	0.000000000000	0.000032023551
0.000000000000	0.000000000000	-0.000032023551

5

-0.000000000000	-0.000000000000	0.000000000000
0.000490470799	0.000490470714	0.000490470881
-0.000490470799	-0.000490470714	0.000490470881
0.000490470799	-0.000490470714	-0.000490470881
-0.000490470799	0.000490470714	-0.000490470881

Note that your output may have very minor numerical differences.

Geometry optimization

Geometry optimization of closed-shell molecules in electronic ground state is as simple as running single point calculations and 4-line MLatom input file, e.g., `opt.inp`, looks like this:

```
AIQM1  
xyzfile=opt.xyz  
optxyz=final_geo.xyz
```

```
geomopt
```

```
optprog=gaussian # or optprog=ase if you choose ASE
```

This input requires `opt.xyz` file with initial XYZ geometries of molecules to be optimized (you can provide many molecules as usual for MLatom), e.g., for hydrogen and methane `opt.xyz` file can look like (geometries in Å):

2

H	0.0000000000	0.0000000000	0.0000000000
---	--------------	--------------	--------------

H	0.7414000000	0.0000000000	0.0000000000
---	--------------	--------------	--------------

5

C	0.0000000000	0.0000000000	0.0000000000
---	--------------	--------------	--------------

H	1.0870000000	0.0000000000	0.0000000000
---	--------------	--------------	--------------

H	-0.3623333220	-1.0248334322	-0.0000000000
---	---------------	---------------	---------------

H	-0.3623333220	0.5124167161	-0.8875317869
---	---------------	--------------	---------------

H	-0.3623333220	0.5124167161	0.8875317869
---	---------------	--------------	--------------

After you prepared your input files `opt.inp` and `opt.xyz`, you can run MLatom as usual:

```
mlatom opt.inp
```

After the calculations finish, the optimized geometries are saved in either Gaussian or ASE-style output files.

If you use Gaussian, the Gaussian output files of geometry optimizations are saved in `mol_1.log`, `mol_2.log`, ... files for each molecule, where you can find the optimized

geometries and visualize them by GaussianView program as in any other typical Gaussian job.

If you use ASE, you can directly obtain optimized XYZ geometries in a single file `final_geo.xyz` with MLatom format, which for our example looks like (geometries in Å):

```

2

H      0.00770082      0.00000000      0.00000000
H      0.73369918      0.00000000      0.00000000
5

C      0.00000000      0.00000000      0.00000000
H      1.08666998     -0.00000000      0.00000000
H     -0.36222332     -1.02452229     -0.00000000
H     -0.36222332      0.51226114     -0.88726233
H     -0.36222332      0.51226114      0.88726233

```

Geometry optimization of transition states structures

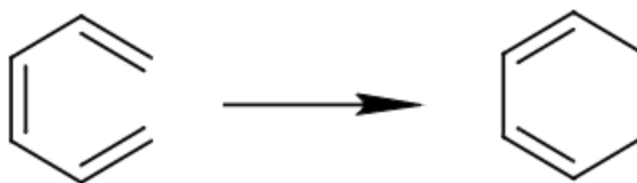
The geometry optimization of transition states structures (TSs) is almost the same as above with the only difference that we replace `geomopt` with `ts` keyword. These calculations require calculations of Hessians and thus are more expensive. Currently, they can only be performed with Gaussian interface, i.e., use `optprog=gaussian` in the input file. Example of input file `opt.inp`:

```

AIQM1
xyzfile=opt.xyz
optxyz=final_geo.xyz
ts
optprog=gaussian

```

The xyz file `opt.xyz` required by the input file should be the initial guess for TS. For a pericyclic reaction:



the file `opt.xyz` can look like:

14

C	-0.12282076	-1.09054053	1.14065341
C	0.17199635	0.23301894	1.48248953
C	-0.12282076	1.35549534	0.70296425
C	-0.12282076	1.35549534	-0.70296425
C	0.17199635	0.23301894	-1.48248953
C	-0.12282076	-1.09054053	-1.14065341
H	0.82454258	0.40945200	2.34048883
H	-0.03331673	2.33317669	1.17510642
H	-0.03331673	2.33317669	-1.17510642
H	0.82454258	0.40945200	-2.34048883
H	0.43031244	-1.88602881	-1.64114758
H	-1.14789312	-1.35457363	-0.93625533
H	0.43031244	-1.88602881	1.64114758
H	-1.14789312	-1.35457363	0.93625533

When the `opt.inp` and `opt.xyz` are both ready, the TS geometry optimization could be conducted via running MLatom as usual:

```
mLatom opt.inp
```

After the calculations finish, the optimized TS geometries will be saved in the XYZ output file `final_geo.xyz`. You can check the TS optimization by investigating frequencies: the number of imaginary frequencies for successful TS optimization should be one and analyze the direction of nuclear motions by visualizing Gaussian output `.log` files with your favorite software (e.g., GaussView).

14

```

6  -0.14824052  -1.18232681   1.09011331
6   0.27697295   0.08303553   1.40719277
6  -0.14824052   1.26264088   0.68226088
6  -0.14824052   1.26264088  -0.68226088
6   0.27697295   0.08303553  -1.40719277
6  -0.14824052  -1.18232681  -1.09011331
1   1.16206956   0.21862475   2.03298866
1  -0.22714441   2.21025461   1.20566513
1  -0.22714441   2.21025461  -1.20566513
1   1.16206956   0.21862475  -2.03298866
1   0.37749585  -2.04342303  -1.49163954
1  -1.19537249  -1.36555399  -0.92796582
1   0.37749585  -2.04342303   1.49163954
1  -1.19537249  -1.36555399   0.92796582

```

Calculation of thermochemical properties

Thermochemical properties of closed-shell molecules in electronic ground state can be calculated at AIQM1 level by adding an option `freq` to the MLatom input file, e.g., `freq.inp` and they are typically run on AIQM1-optimized geometries. An example of MLatom input file:

```

AIQM1
xyzfile=freq.xyz

```

freq

optprog=gaussian # or optprog=ase if you choose ASE

This input requires `freq.xyz` file with initial XYZ geometries of molecules (you can provide many molecules as usual for MLatom), e.g., for hydrogen and methane `freq.xyz` file can look like (geometries in Å):

```

2

H      0.000000      0.000000      0.363008
H      0.000000      0.000000     -0.363008
5

C      0.000000      0.000000      0.000000
H      0.627580      0.627580      0.627580
H     -0.627580     -0.627580      0.627580
H      0.627580     -0.627580     -0.627580
H     -0.627580      0.627580     -0.627580

```

After you prepared your input files `freq.inp` and `freq.xyz`, you can run MLatom as usual:

```
mlatom freq.inp > freq.out
```

After the calculations finish, MLatom output `freq.out` will contain the summary with atomization enthalpy at 0 K, ZPVE-exclusive atomization energy at 0 K, and heat of formation at 298 K for each molecule.

If you use Gaussian, the Gaussian output files of frequency calculations are saved in `mol_1.log`, `mol_2.log`, ... files for each molecule; these files contain ZPVE energy and lots of thermochemical data such as entropy and the Gibbs free energy. MLatom output will contain additional information as described above with following lines for hydrogen and methane example:

```

Standard deviation of NN contribution      :      0.00892407 Hartree
5.59994 kcal/mol
NN contribution                           :      -0.00210898 Hartree
Sum of atomic self energies               :      -0.08587317 Hartree
ODM2* contribution                       :      -1.09094119 Hartree
D4 contribution                           :      -0.00000889 Hartree
Total energy                             :      -1.17893224 Hartree
Atomization enthalpy at 0 K               :      106.17239 kcal/mol
ZPE exclusive atomization energy at 0 K   :      111.17678 kcal/mol
Heat of formation at 298.15 K             :      -2.85652 kcal/mol
* Warning * Heat of formation have high uncertainty!
Standard deviation of NN contribution      :      0.00025608 Hartree
0.16069 kcal/mol
NN contribution                           :      0.00958812 Hartree
Sum of atomic self energies               :      -33.60470494 Hartree
ODM2* contribution                       :      -6.86968756 Hartree
D4 contribution                           :      -0.00010193 Hartree
Total energy                             :      -40.46490632 Hartree
Atomization enthalpy at 0 K               :      391.58894 kcal/mol
ZPE exclusive atomization energy at 0 K   :      419.90907 kcal/mol
Heat of formation at 298.15 K             :      -17.30543 kcal/mol

```

When you use ASE for the calculation of thermochemical properties, you should specify `ase.linear` and `ase.symmetrynumber` these two keywords. `ase.linear` is 0 for nonlinear molecule, 1 for linear molecule, and `ase.symmetrynumber` is the rotational symmetry number for each molecule, (see Table 10.1 and Appendix B of C. Cramer “Essentials of Computational Chemistry”, 2nd Ed.). For example, for hydrogen and methane these two molecules, you should set `ase.linear=1,0` and `ase.symmetrynumber=2,12`.

If you use ASE, MLatom output will contain the same lines as above, but also include additional data such as entropy and the Gibbs free energy.

Beyond closed-shell molecules in ground state

If we go beyond closed-shell molecules in their ground-state, you need to tell MLatom the charge and multiplicity and/or what excited-state properties you want to calculate. Since this information will change the semiempirical quantum mechanical part of AIQM1, you have to inform the MNDO program to do the corresponding calculations. MLatom implementation therefore supports the request to read user-defined MNDO keywords via option `mndokeywords=[file name with MNDO keywords]`, which are passed to the MNDO program. Please consult the MNDO program manual for the available keywords. Note: whenever you request special keywords for MNDO, keywords `iop=-22 immdp=-1 nsav15=3 igeom=1 iform=1` are always required and AIQM1 calculations can be run only for a single molecule in xyz file.

Below we show several typical examples for calculating charged species and excited-state properties.

Calculations of charged species and radicals

For example, if we want to optimize geometry of protonated water H_3O^+ , then you can use the following MLatom input `opt.inp`:

```
AIQM1
xyzfile=opt.xyz
optxyz=final_geo.xyz
geomopt
optprog=ase
charges=1
```

This input requires `opt.xyz` file with initial XYZ geometries of molecules to be optimized, e.g. (geometries in Å):

```
4

O      0.00000000    0.00000000    0.08727273
H      0.00000000    0.90509668   -0.23272727
```

<i>H</i>	<i>-0.78383672</i>	<i>-0.45254834</i>	<i>-0.23272727</i>
<i>H</i>	<i>0.78383672</i>	<i>-0.45254834</i>	<i>-0.23272727</i>

After you prepared your input files `opt.inp` and `opt.xyz`, you can run MLatom as usual:

```
mlatom opt.inp
```

The output geometry is saved in `final_geo.xyz` with MLatom format, which for our example looks like (geometries in Å):

```
4

O      0.00000000    -0.00000000    0.12401796
H      0.00000000    0.90385318    -0.24497568
H     -0.78275982    -0.45192659    -0.24497568
H      0.78275982    -0.45192659    -0.24497568
```

When you want to calculate the heat of formation of H_3O^+ with the above optimized geometry, then you can use the following MLatom input `freq.inp`:

```
AIQM1
xyzfile=final_geo.xyz
freq
optprog=ase
charges=1
```

After you run MLatom by `mlatom freq.inp > freq.out`, you can get the following heat of formation from the output:

```
Heat of formation at 298.15 K      :      150.84947 kcal/mol
```

** Warning * Heat of formation have high uncertainty!*

Vertical excitation energies

For example, if we want to calculate the vertical excitation energies of ethene in $^1B_{1u}$ state, then you can use the following MLatom input `vee.inp`:

```
AIQM1
xyzfile=vee.xyz
mndokeywords=mndokw
```

This input requires a `vee.xyz` file with XYZ geometries of molecules, e.g. (geometries in Å):

```
6
Ethene 1B1u
C      0.000000      0.000000      0.668188
C      0.000000      0.000000     -0.668188
H      0.000000      0.923274      1.238289
H      0.000000     -0.923274      1.238289
H      0.000000      0.923274     -1.238289
H      0.000000     -0.923274     -1.238289
```

and `mndokw` file with MNDO keywords:

```
iop=-22 immdp=-1 +
igeom=1 iform=1 +
jop=-2 nsav15=3 +
kci=5 ici1=1 ici2=1 iroot=2 ioutci=2 +
movo=-1 nciref=1 mciref=3 levexc=2
```

After you prepared your input files `vee.inp`, `vee.xyz`, and `mindokw` you can run MLatom as usual:

```
mlatom vee.inp
```

The output of vertical excitation energy is saved in the MNDO program outfile `mindo.out`, where you can see:

```
State 2, Mult. 1, B1u (4), E-E(1)= 7.910791 eV, E=
-304.054934 eV
```

Excited-state geometry optimization

For example, if we want to optimize geometry of formaldehyde in $1n\pi^*$ excited state ($1A''$), then you can use the following MLatom input `opt.inp`:

```
AIQM1
xyzfile=opt.xyz
optxyz=final_geo.xyz
geomopt
optprog=ase
mndokeywords=mndokw
```

This input requires `opt.xyz` file with initial XYZ geometries of molecules to be optimized, e.g. (geometries in Å):

```
4
formaldehyde 1npi*
C      -0.02667227      0.64339915     -0.00000000
O      -0.02667227     -0.71674790      0.00000000
```

<i>H</i>	<i>0.18670592</i>	<i>0.93679416</i>	<i>1.04362466</i>
<i>H</i>	<i>0.18670592</i>	<i>0.93679416</i>	<i>-1.04362466</i>

and `mndokw` file with MNDO keywords:

```
iop=-22 immdp=-1 +
igeom=1 iform=1 +
jop=-2 nsav15=3 +
kcharge=0 imult=0 +
kci=5 ici1=6 ici2=4 jci1=1 jci2=1 ncisym=2 +
movo=-1 nciref=1 mciref=3 levexc=2 iroot=1 lroot=1
```

The first line in `mndokw` file requests the use of the ODM2* Hamiltonian (you do not need to modify it), the second line specifies that geometry is given in a free XYZ format, the third line requests calculation of gradients and saving energies and gradients into `fort.15` file (required for geometry optimization), the fourth line sets charge (`kcharge=0`) and multiplicity (`imult=0`), and the following lines setup the type of excited-state calculations. `kci=5` requests GUGA-CI approach for excited state calculation; `ici1` and `ici2` define the number of active occupied orbitals and unoccupied orbitals respectively; `jci1` and `jci2` define the number of occupied and unoccupied pi-MOs included in the active space; `ncisym` define the state symmetry. Here the initial geometry has C_s point group, which has A' and A'' this two irreducible representation, and we use `ncisym=2` to calculate the $^1A''$ state. `nciref` and `mciref` define the number of reference occupations and definition of reference occupations respectively; `levexc` define the maximum excitation level; `iroot` and `lroot` is the number of lowest CI states computed and the number of the CI state of interest.

After you prepared your input files `opt.inp`, `opt.xyz`, and `mndokw` you can run MLatom as usual:

```
mlatom opt.inp
```

The output geometry is saved in `final_geo.xyz` with MLatom format, which for our example looks like (geometries in Å):

```
4

C      -0.10906735      0.55934608     -0.00000000
O      -0.02784720     -0.77782274      0.00000000
H       0.22849093      1.00935811      0.92370071
H       0.22849093      1.00935811     -0.92370071
```

Visualizing molecular orbitals

If you use Sparrow for providing the ODM2* part:

```
AIQM1
qmprog=sparrow
xyzfile=sp.xyz
yestfile=enest.dat
QMprogramKeywords=SparrowKW
```

You need to use `QMprogramKeywords` for printing orbitals in Molden wavefunction format. The content of the file `SparrowKW` should be `-W`. After calculation, the `wavefunction.molden.input` file will be generated. You can use [Jmol](#) or [MOLDEN](#) program to open this file and see the orbitals. For Jmol program, to get the pictures of orbitals. You should right-click in the panel and select `Surface` and `M0`. For MOLDEN program, to get the pictures of orbitals, you need to choose `Dens.` `Mode` from the control panel and select the `Orbital` button, then select the `Space` option to set the contour value. Besides, you can also use Chemcraft to visualize the orbitals, but you need to rename this file with a `.molden` suffix.

If you use MNDO for providing the ODM2* part:

```
AIQM1
qmprog=mndo
xyzfile=sp.xyz
yestfile=enest.dat
mndokeywords=mndokw
```

You should provide the `mndokw` file with general MNDO keywords and add an extra keyword `nsav13=2`:

```
iop=-22 immdp=-1 +
igeom=1 iform=1 +
jop=-1 nsav15=3 +
kharge=0 imult=0 nsav13=2
```

Then the `molten.dat` file will be generated. You can also use Jmol or MOLDEN program to visualize the orbitals.

Citations

If you have used AIQM1, then the following citations are appropriate in your publication:

- **AIQM1:** P. Zheng, R. Zubatyuk, W. Wu, O. Isayev, P. O. Dral, [Artificial Intelligence-Enhanced Quantum Chemical Method with Broad Applicability](#), *Nat. Commun.*, **2021**, 12, 7022. DOI: 10.1038/s41467-021-27340-2.
- **MLatom:** Pavlo O. Dral, Fuchun Ge, Bao-Xin Xue, Yi-Fan Hou, Max Pinheiro Jr, Jianxing Huang, Mario Barbatti, [MLatom 2: An Integrative Platform for Atomistic Machine Learning](#). *Top. Curr. Chem.* **2021**, 379, 27. DOI: 10.1007/s41061-021-00339-5.
- **MLatom:** Pavlo O. Dral, Peikun Zheng, Bao-Xin Xue, Fuchun Ge, Yi-Fan Hou, Max Pinheiro Jr, *MLatom: A Package for Atomistic Simulations with Machine Learning*, version 2.0.1beta. Xiamen University, Xiamen, China, **2013–2021**. <http://MLatom.com>.

- **ODM2* Hamiltonian:** P. O. Dral, X. Wu, W. Thiel, *J. Chem. Theory Comput.* **2019**, 15, 1743.
 - If you use MNDO program:

MNDO program: W. Thiel, MNDO [check your version](Max-Planck-Institut für Kohlenforschung, Mulheim an der Ruhr, [year])

- If you use MLatom@XACS cloud computing service, where SCINE Sparrow is used:

F. Bosia, T. Husch, C. H. Müller, S. Polonius, J.-G. Sobez, M. Steiner, J. P. Unsleber, A. C. Vaucher, T. Weymuth, M. Reiher, qcscine/sparrow: to-be-released alpha version with modifications by P. Zheng, **2022**

T. Husch, A. C. Vaucher, M. Reiher, *Int. J. Quantum Chem.*, **2018**, 118, e25799

- **D4:** E. Caldeweyher, C. Bannwarth, S. Grimme, *J. Chem. Phys.* 2017, 147, 034112
- **D4 program:** E. Caldeweyher, S. Ehlert, S. Grimme, DFT-D4, Version [check your version], (Mulliken Center for Theoretical Chemistry, University of Bonn, [year])
- **ANI model:** J. S. Smith, O. Isayev, A. E. Roitberg, *Chem. Sci.* 2017, 8, 3192
- **TorchANI program:** X. Gao, F. Ramezanghorbani, O. Isayev, J. S. Smith, A. E. Roitberg, *J. Chem. Inf. Model.* **2020**, 60, 3408

6 Comments on “AIQM1”



Igor

August 5, 2022 at 3:07 pm

It is a very interesting approach to calculation of some molecular properties.

I tried to use the AIQM1 approach to calculate the heat of formation for closed-shell CHNO compounds in our cloud platform, however, I faced the problem that all my thermochemical jobs crash with the following error:

Traceback (most recent call last):

File “/home/xmvp/.local/lib/python3.8/site-packages/MLatom/MLatom.py”, line 197, in

```
MLatomMainCls()
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/MLatom.py", line 118, in  
__init__
```

```
MLtasks.MLtasksCls(argsMLtasks = args.args2pass)
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/MLtasks.py", line 100, in  
__init__
```

```
geomopt.geomoptCls(args.args2pass)
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/geomopt.py", line 49, in  
__init__
```

```
self.do_geomopt()
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/geomopt.py", line 62, in  
do_geomopt
```

```
self.ase_freq()
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/geomopt.py", line 241, in  
ase_freq
```

```
energy, ZPE, H298 = thermocalc(self.atoms[i], linear[i], sn[i], mult)
```

```
File "/home/xmvp/.local/lib/python3.8/site-packages/MLatom/thermo.py", line 113, in  
thermocalc
```

```
thermo = IdealGasThermo(vib_energies=vib_energies,
```

```
File "/home/xmvp/.local/lib/python3.8/site-  
packages/mlatom3rd/anaconda3_20220204/lib/python3.8/site-  
packages/ase/thermochemistry.py", line 453, in __init__
```

```
raise ValueError('Imaginary frequencies are present.')
```

```
ValueError: Imaginary frequencies are present.
```

I perform calculation steps according to these instructions. Tell me please, what could be the problem?

[↩ Reply](#)



Pavlo Dral

August 8, 2022 at 9:57 am

there is apparently a problem that your structures are not true minima and have imaginary frequencies. It would be helpful if you could provide an example of input that causes this error. Did you optimize your structure before running freq calculations? You can also join our Slack channel where we can help faster:

https://join.slack.com/t/xacs-support/shared_invite/zt-1d42zcskn-vFXZrHk3GxZ5ZDovkedE4g

 Reply



Gang_Pao

September 20, 2023 at 9:08 am

Why do I lack energy data in the log file after using the AIQM1 method for geomopt?

 Reply



Pavlo Dral

September 21, 2023 at 6:20 am

could you please provide more details?

What version of MLatom do you use, etc.?

Also, please consider joining our Slack workspace (https://join.slack.com/t/xacs-support/shared_invite/zt-1gm1lpn68-pReQhfYGu813eCqwmvdGvA) or WeChat user group (inquire the QR code via email to dral at xmu.edu.cn) for faster communication.

 Reply



cyy

October 4, 2023 at 4:18 am

Sorry I have a stupid question. When training AIQM1, you fitted NN using the ANI-1x and ANI-1ccx data sets. "For 4.6M geometries of the ANI-1x data set, ω B97X/def2-TZVPP energies and forces are available." Does it mean there are 4.6M different molecules in the data set?

 Reply



Pavlo Dral

October 7, 2023 at 8:11 am

We took ANI-1x and ANI-1ccx data sets as described in literature:

<http://doi.org/10.1038/s41597-020-0473-z>. Basically, there are thousands of molecules with different conformations – so it is not 4.6M different molecules.

 Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

E-mail *

Website

POST COMMENT

Search here ...



NEWS & POSTS

[MACE-OFF](#) is available on Aitomistic Hub

[npj Comput. Mater.: Efficient Machine Learning Protocol For Accelerating Trajectory Surface Hopping Dynamics](#)

[Meet OMNI-P2x — the First Universal ML Potential for Excited States!](#)

[Aitomia: Your Intelligent Assistant for AI-Driven Atomistic and Quantum Chemical Simulations](#)

[Postdoc for AI-enhanced atomistic simulations in Dral's group](#)

SUBSCRIBE VIA EMAIL

Please subscribe to receive News & Posts

SUBSCRIBE

Join 40 other subscribers

FOLLOW ME ON TWITTER

[My Tweets](#)

Copyright © 2025 Dr. Pavlo O. Dral | Powered by Responsive
Theme